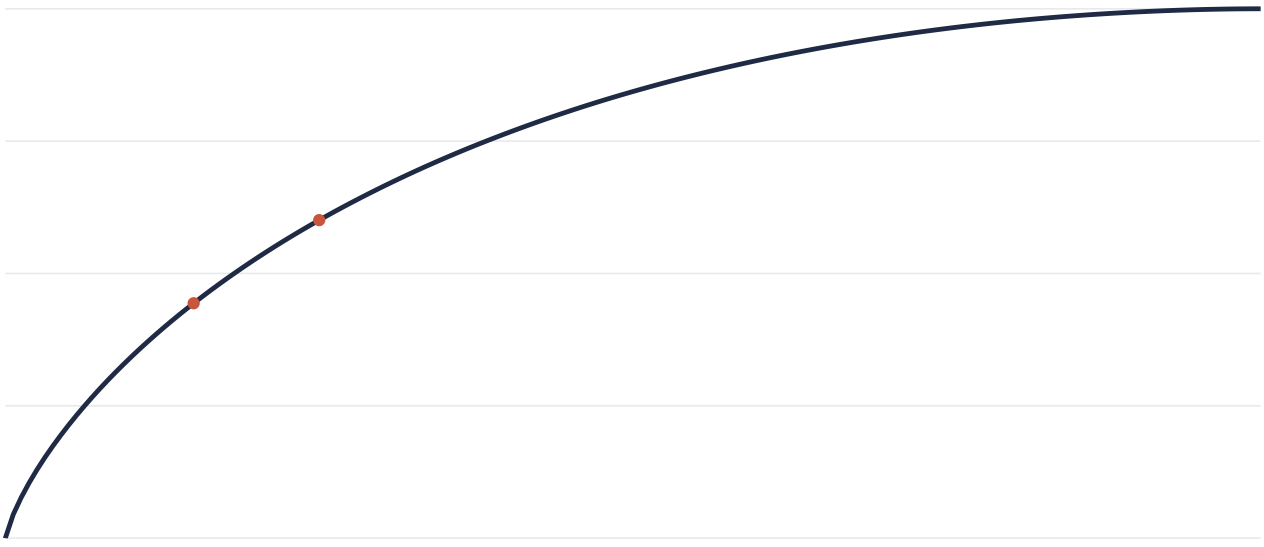
BUSINESS PROPOSAL

Keeping the customers who are about to leave

Churn prediction and targeted retention for a telecom customer base



PREPARED BY

Karen Maher

Independent Data Science Consultant

PREPARED FOR

Executive management and commercial team

DATE

October 2026

Version 1.0, confidential

A short note first

Why I wrote this, and how to read it.

Dear team,

Most operators I have looked at can tell you their churn rate to two decimal places. Very few can tell you which customers will leave next month, and fewer still which of those departures will actually hurt.

That gap is what this proposal is about. You already hold the raw material: a client file and a usage file. For past customers they also show who left, so I can learn from them. Then I score the customers with no churn flag yet: each gets a probability of leaving and a score for what losing them would cost. That list tells you who deserves an offer first.

I have kept the theory short and tried to be specific about what you will receive, how it gets built and how I would expect you to use it. Where I am making an assumption, I say so. The financial figures in sections 09 to 11 come from a simulated population of 100,000 subscribers, not from your data. They show the logic of the plan, and the real numbers get rebuilt in the first two weeks.

Thank you for taking the time to read it.

Karen Maher
Independent Data Science Consultant

Contents

01	Executive summary	3
02	The cost of churn	4
03	Your data	5
04	How the solution works	6
05	Data preparation and features	7
06	Modeling and validation	8
07	The Expected Loss Score	9
08	The FinalDataframe.csv deliverable	10
09	Who gets an offer	11
10	Financial impact	12
11	Word of mouth and sensitivity	13
12	Timeline and risks	14
13	Next steps	15

01

Executive summary

One list, ranked by what you stand to lose.

Your customers do not leave all at once, and they do not all matter equally. A small group of them, the ones with a high chance of leaving and a high value to the business, accounts for a large share of what you lose every year. In your systems today those customers look the same as everyone else.

I teach a model what leaving looks like using **Client.csv** and **Record.csv** with the churn column included. You then send the same two files without churn, for the customers to score. The model estimates each one's probability of leaving, and I multiply it by the customer's value to get an **Expected Loss Score**: how much money is at risk. Everything comes back as **FinalDataframe.csv**, one row per customer ID.

The plan for using it is deliberately narrow. Offers go to the customers in the **top 15% to 25%** of the Expected Loss ranking and to nobody else. That keeps the discount budget small and points it where a saved account is worth the most.

Profit is protected in two ways. First, the offer costs less than the margin it saves as long as it reduces churn in that group by roughly 15% or more, which is the break-even point (section 10). Second, a customer who gets a good offer tends to talk about it, and every referred customer adds margin on top. I treat that second effect as upside and never as the reason to go ahead.

At a glance	
Customers in the offer group	20,000
Share of expected loss they hold	53%
Break-even cut in churn	15%
Net margin, retention only	+$173k
Net margin with referrals	+$225k
Time to FinalDataframe.csv	8 weeks

Illustrative: simulated base of 100,000 subscribers, 20% offer group. Not your data.

What I need from you

- The two files with churn included, so the model can learn from past customers.
- Later, the same two files without churn, for the customers to score.
- The exact definition of churn, and a margin figure per plan if you have one.
- One person who can decide the offers and approve a holdout group.

> *The model is the easy part to describe and the least interesting. What decides whether this pays off is the second half: a sensible way of choosing who gets an offer, and a way of measuring whether the offer worked.*

02

The cost of churn

Losing customers is expensive. The usual responses are often more expensive.

Every subscriber who leaves takes future margin with them, and replacing that subscriber means paying acquisition costs again: advertising, handset subsidies, dealer commissions, activation promotions. It is widely accepted in the industry that winning a customer costs considerably more than keeping one. I will not quote a ratio here, because the right one is yours, and I will calculate it from your own acquisition and retention figures during the data audit.

The harder problem is what happens in between. Most retention programs I have seen fall into one of three patterns.

Approach	How it works	Where it goes wrong
Blanket discount	Everyone, or a very large segment, gets the same offer.	Most of the base was never leaving. You pay a discount to all of them and margin shrinks across the board.
Cancellation desk	A save team calls when the customer asks to leave.	The decision is already made. Save rates are low and the offers tend to be the most generous you have.
Scored targeting (this proposal)	Offers go to customers ranked by probability of leaving times value.	Needs a model you can trust and a holdout group to measure the effect. That is the work I am proposing.

> *A discount given to a customer who was staying anyway is not a retention cost. It is a price cut.*

Why probability alone is not enough

A customer with an 85% chance of leaving who pays for the cheapest plan may cost you less than a customer with a 35% chance who holds three lines and a family plan. A list sorted by probability sends the budget to the first customer. A list sorted by expected loss sends it to the second. Section 07 has a worked example.

What this means for the design

- The model has to produce **honest probabilities**, not just a ranking, because they get multiplied by money.
- The ranking has to include **value**, so the budget goes where a saved customer is worth the most.
- The result has to be **testable**: some customers in the target group get no offer, so you can see what the offer really did.

03

Your data

Two files, two rounds: one to teach the model, one to score.

File	Columns	What it holds
Client.csv	50	Demographic and personal information about each customer: income, region, marital status, number of phones, and similar fields.
Record.csv	51	Usage and plan records: average minutes, revenue, dropped calls, plan details. The training copy also has the churn indicator.

I train on the files with the churn column, then score the same files without it. Six things need to be true for this to work, and I would rather confirm them in week one than find out in week five.

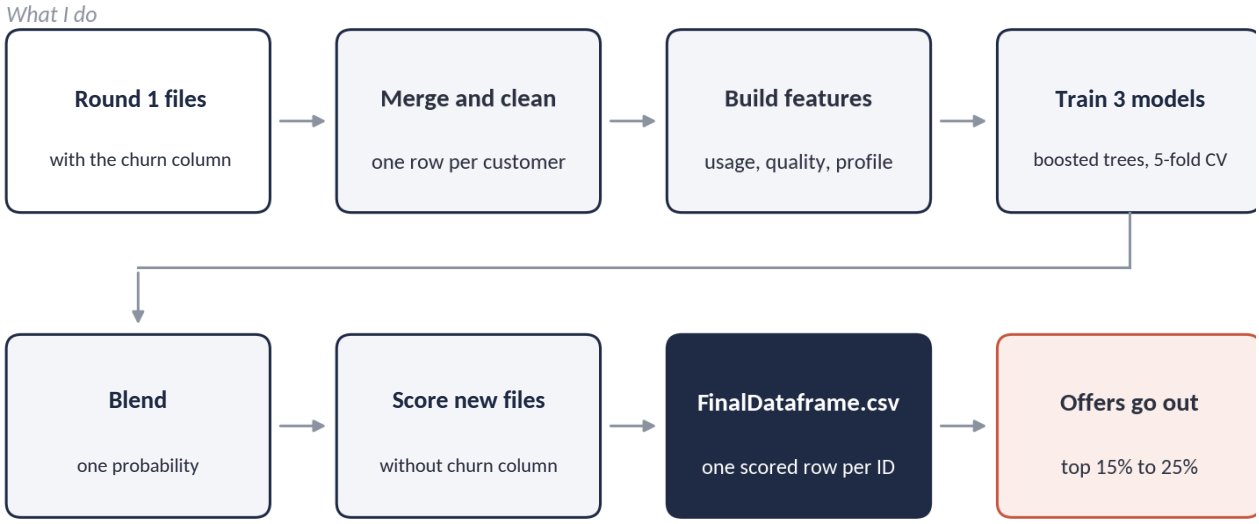
Question	Why it matters
1. Is there a shared customer ID, and is it unique in each file?	Without a clean one-to-one join I cannot build one row per customer, and the final file has no key to come back on.
2. What exactly does the churn flag mean?	Left within 30, 60 or 90 days? Voluntary or forced? The model predicts whatever the label says, so the label has to match the decision you want to make.
3. Does any column record something that happens after a customer decides to leave?	Fields such as a final-month usage collapse or a cancellation reason would make the model look excellent in testing and useless in practice. I remove them.
4. Will the scoring files have the same columns, without churn?	The model only works if both rounds look alike. Different columns or codes would make the scores unreliable.
5. Is there a date or period column?	If so, I train on earlier periods and test on later ones. That is the honest way to measure a model that will score next month's customers.
6. Is a margin or cost-to-serve figure available?	Expected Loss needs a value per customer. Margin is better than revenue, and revenue is better than nothing.

> *Privacy. Customer identifiers can be hashed before the files leave your environment. All I need is the same key back at the end, so scores can be matched to customers on your side.*

04

How the solution works

Learn from past customers, then score the current ones.



	Step	What happens
1	Collect and clean	Round 1. You send Client.csv and Record.csv with the churn column. I join them on Customer_ID, clean them and log every rule.
2	Build features	Turn raw columns into about 70 signals the model can use: trends, ratios, quality rates, tenure and peer comparisons.
3	Train the models	LightGBM, XGBoost and CatBoost learn from the past customers, checked with 5-fold cross-validation before they score anyone.
4	Blend	Combine the three predictions with weights 4 : 2 : 1 into one churn probability. Out-of-fold ROC-AUC is 0.7063.
5	Score new files	Round 2. You send the same two files without churn. The model gives each customer a probability of leaving, times customer value.
6	Deliver and monitor	FinalDataframe.csv, the model, the documentation, and a monthly check on whether it still works.

> *The steps are conventional on purpose. The value is not in a clever algorithm. It is in careful validation, and in connecting the output to a decision your commercial team can actually make on Monday morning.*

05

Data preparation and features

Most of a churn model's accuracy is decided before the model is chosen.

Cleaning is unglamorous and it takes the first two to three weeks. The rules are simple. Every dropped column is logged. Missing values stay missing, because tree models handle them and a missing field can itself be a signal. Rare categories are grouped. From the roughly 100 raw columns I build about 70 extra features, then remove any that add nothing. The scoring files go through exactly the same steps.

Feature families

Family	Examples	Why it should matter
Tenure and lifecycle	Months as a customer, handset age, phone changes per year	Risk is rarely flat over a customer's life. It tends to jump at specific moments.
Usage level and trend	Recent against earlier minutes, quantity and revenue, and the relative change	A customer who talks less than they used to is often halfway out already.
Revenue and billing	Revenue per minute, overage, share of data and roaming	Bill shock and unusual changes show up here first.
Service quality	Dropped, blocked and unanswered call rates, care calls	A bad experience is a leading signal, and it is one you can fix.
Plan and device	Handset age, lines, add-on services	Shapes what an alternative offer can look like.
Household and profile	Income band, region, marital status, number of phones	Context rather than cause. Used with care, see below.

On top of these, I compare each customer with the average of their own area and credit class. The data dictionary I hand over will list every feature I built, every one I dropped, and the reason.

> *A note on fairness. Some personal attributes, marital status or region for example, can stand in for sensitive groups. I will test whether they add real predictive value. If they do not, they stay out. If they do, I report how much they move the score so your team can decide whether to keep them.*

06

Modeling and validation

Honest probabilities matter more than a high accuracy score.

Models. I use three tree-based models: LightGBM, XGBoost and CatBoost. They are the standard tools for mixed customer data like yours and they cope well with missing values. LightGBM carries the most weight in the final blend.

Validation. I use five-fold stratified cross-validation. The customers are split into five parts, each part is predicted by models trained on the other four, and churners and non-churners are kept in the same proportion in every part.



Illustrative: one of the five rounds. Every part takes a turn as the held-out part.

Blend. The three models are combined with weights 4 : 2 : 1, so LightGBM contributes about 57% of the final probability, XGBoost 29% and CatBoost 14%. The blend is steadier than any single model because each one makes slightly different mistakes.

Scoring new customers. After validation, the models are retrained on all past customers with a known outcome. They then score the files without churn, so every customer in FinalDataframe.csv is new to the model, just like in real use.

Result. The blend reaches an out-of-fold ROC-AUC of 0.7063. Pick one customer who left and one who stayed, and the model gives the leaver the higher risk about 71 times in 100.

Checks before rollout

Measure	What I report	Result or bar
ROC-AUC	Out-of-fold, across all five folds	0.7063 so far
Hold-out check	Score on data not used for screening or tuning	Close to 0.7063
Lift in the top 20%	Churn rate in the top 20% against the overall rate	Clearly above 1.0
Stability	Same checks by region and plan type	No segment clearly worse than average

> *An AUC of 0.7063 is moderate. It is good enough to rank customers and focus a budget, and not good enough to say that one particular customer will leave. That is why offers go to a band of customers and why the pilot keeps a holdout group.*

07

The Expected Loss Score

The number that decides who gets an offer.

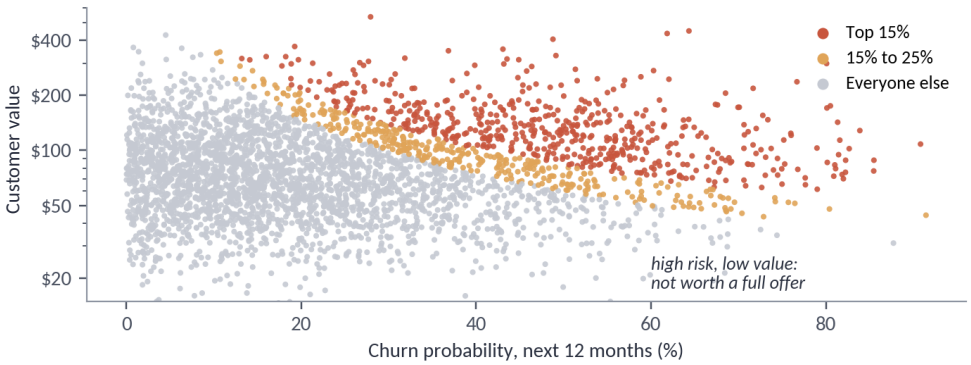
Expected_Loss_Score = churn_probability × Customer_Value

Customer_Value is the customer's total revenue (totrev in Record.csv), the money they have already brought in. It stands in for future value, and if you have a margin or lifetime value figure I can use that instead. The score is in currency, so it reads as money at risk.

Customer	Churn probability	Customer value	Expected Loss	Rank by probability	Rank by Expected Loss
A	82%	$38	$31	1	5
B	35%	$210	$74	4	1
C	64%	$95	$61	2	3
D	12%	$320	$38	5	4
E	51%	$140	$71	3	2

Five customers, invented for illustration.

Customer A is the most likely to leave and the least costly of the five to lose. A probability-only list puts A first and B fourth. The score reverses that.



Simulated customers. The offer group sits in the upper right: likely to leave and worth keeping.

Tier	Position in the ranking	Suggested action
Priority	Top 15%	Retention offer plus a personal contact
Secondary	Next 10% (15% to 25%)	Lighter retention offer
Holdout	Part of the offer group	Gets nothing, so the real effect can be measured
Everyone else	Below 25%	No offer, normal service

08

The FinalDataframe.csv deliverable

What lands on your desk, and how to use it.

The deliverable is one file with one row per customer in the scoring round, keyed on the same **Customer_ID** as the files you send, so it joins straight back into your CRM or reporting tool. It has exactly three columns.

- **Customer_ID:** the same ID used in Client.csv and Record.csv.
- **churn_probability:** a number between 0 and 1, the model's estimate that this customer leaves. Section 06 explains how it is trained and checked.
- **Expected_Loss_Score:** churn_probability multiplied by the customer's value, see section 07. The higher it is, the more money is at risk.

How to use it

- The file is sorted by Expected_Loss_Score, highest first. The top of the list is where to start.
- Offers go to the top 15% to 25% of the list and to nobody else. The first 15% is the priority group, and the next 10% gets a lighter offer.
- Keep both numbers in view. churn_probability says how likely the customer is to leave, and Expected_Loss_Score says how much it would cost you.

Also included

- The trained model and a scoring script, so you can re-score next month's customers yourselves.
- A data dictionary and the data-preparation log, listing every rule applied.
- A methodology and validation report: the evidence behind the acceptance criteria.
- A one-page guide for the retention team on how to read the two scores and who gets an offer.

> *Once the pipeline exists, re-scoring is a matter of minutes. I suggest refreshing the file monthly, and retraining the model every six months or whenever monitoring shows it has drifted.*

09

Who gets an offer

Why the top 15% to 25%, and how to pick the number inside that range.



Simulated base of 100,000 subscribers. The curve shows how fast expected loss accumulates as you move down the ranking.

Offer group	Customers per 100,000	Share of expected loss	Avg churn probability	Avg customer value
Top 15%	15,000	44%	48%	$148
Top 20%	20,000	53%	46%	$138
Top 25%	25,000	60%	44%	$131

The curve flattens as you go down the list. In this simulation the first 15% holds 44% of expected loss and the next 10% adds only 16 percentage points. Moving from 15% to 20% adds about $12k of net retention margin per 100,000 subscribers; moving from 20% to 25% adds about $3k. Beyond that point each extra offer costs more than the margin it brings back.

My suggestion is to start at the 15% line, run it against a holdout, and widen to 20% or 25% only if the measured results clear the break-even point.

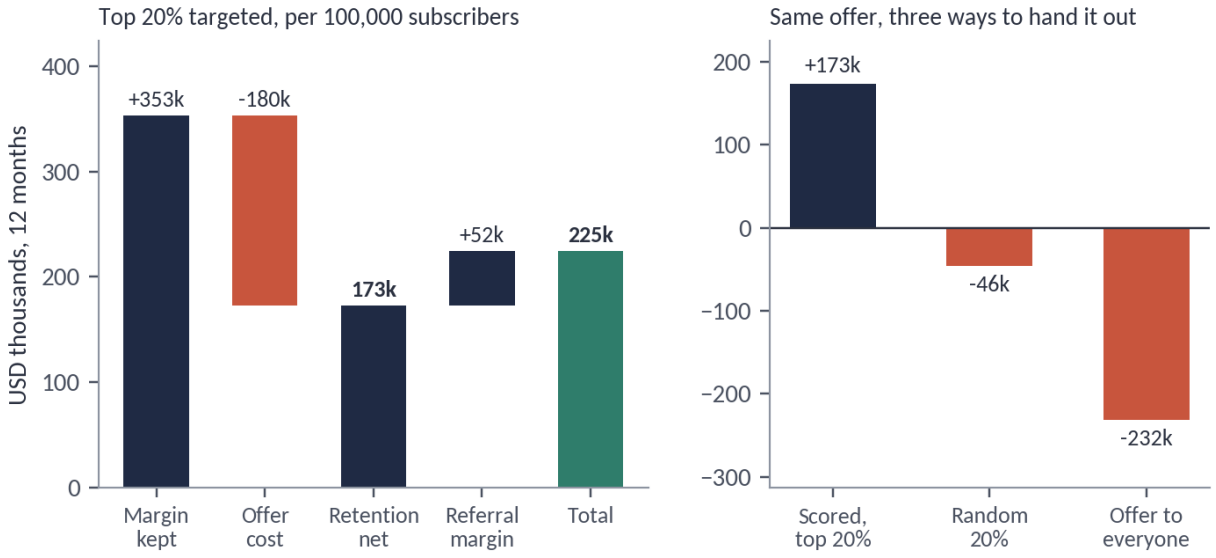
Measuring it properly

Inside the offer group, hold back a random 10% who receive nothing. The difference in churn between the two halves is the real effect of the offer, measured and not estimated. Without it, every improvement in churn gets credited to the program whether or not the program caused it, and the next budget discussion becomes an argument about opinions.

10

Financial impact

Does the profit hold? Per 100,000 subscribers, yes, provided the offer does its job.



Left: how the 20% scenario builds up. Right: the same offer, handed out three different ways.

In this simulation the offer group holds $1.18M of the $2.23M expected loss. The offers cost $180k and bring back $353k of margin, a net of +$173k. The same budget handed out at random loses $46k, and an offer to everyone loses $232k. Targeting is what turns the same offer from a cost into a gain.

Assumption	Value
Average 12-month churn probability	24.6%
Average 12-month customer margin	$91
Offer group	20% of customers, ranked by Expected Loss
Offer cost per recipient	$9, charged for every recipient whether or not they accept
Effect on recipients	Churn probability reduced by 30%, relative
Referrals	6 new customers per 100 offers
Referred customer, first year	$55 margin, less a $12 reward

Break-even. The offer pays for itself when it reduces churn in the group by 13.7% (top 15%), 15.3% (top 20%) or 16.8% (top 25%). The wider the net, the harder the break-even gets, which is one more reason to start narrow.

> *These figures are simulated. The real ones get built in week 6 from your data, your margins and the offers your commercial team actually wants to run.*

11

Word of mouth and sensitivity

The part nobody can promise, shown honestly.

The growth side of this plan rests on a simple chain. A customer who was about to leave gets a fair offer, stays, and tells someone. That is plausible, and it is also the least measurable part of the whole plan, so I handle it conservatively.

How to make it measurable

- Put a referral code or a share link inside the retention offer, so new sign-ups can be traced back.
- Make the offer easy to pass on, for example a bonus the customer can give to a friend as well as keep.
- Report referral results separately from retention results. They answer different questions.

Net margin over 12 months, per 100,000 subscribers, 20% offer group

Churn reduction in the offer group	No referrals	3 per 100 offers	6 per 100 offers	10 per 100 offers
15%	-$3k	+$22k	+$48k	+$83k
20%	+$55k	+$81k	+$107k	+$141k
30%	+$173k	+$199k	+$225k	+$259k
40%	+$291k	+$317k	+$342k	+$377k

Shaded cells lose money. Simulated base, offer cost $9 per recipient.

Read across and referrals move the result by tens of thousands of dollars. Read down and the size of the churn reduction moves it by close to $294k. Retention does most of the work, and referrals sit on top of it.

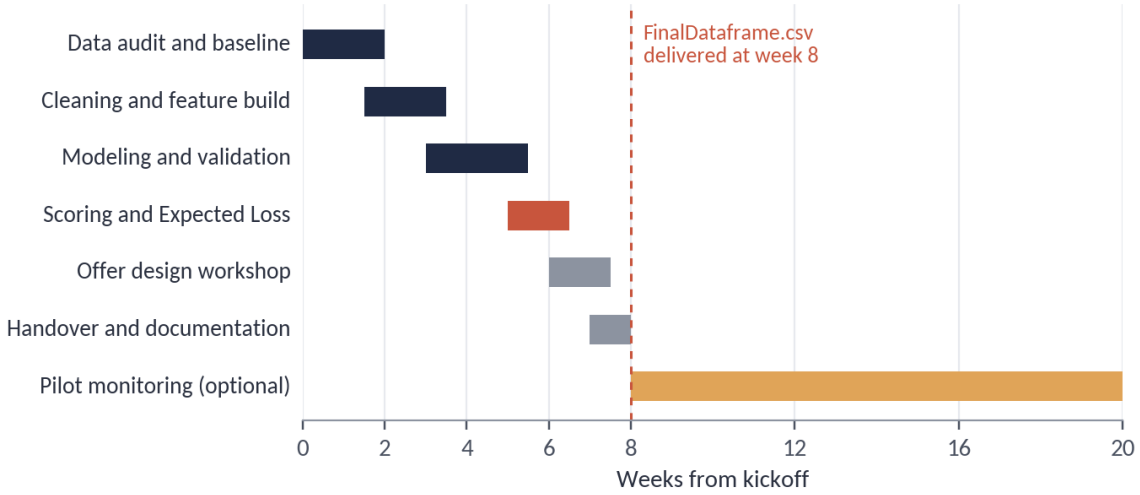
> *If the program only breaks even without referrals, it is still worth running, because the referral upside is free. If it only works with referrals, I would not recommend it. That is the rule I will apply when we review the pilot.*

There is a second, quieter benefit that is hard to put in a table. A customer who was angry enough to consider leaving, and was then treated well, is usually more loyal than one who never had a reason to complain. I would not build the business case on it, but I would not ignore it either.

12

Timeline and risks

Eight weeks to the file, then three months of watching it work.



Week	Milestone
2	Training files received, data audit report and an agreed definition of churn
5	Model comparison and validation results, including ROC-AUC and the hold-out check
6	Scoring files (no churn) received, draft FinalDataframe.csv, Expected Loss sign-off
8	Final delivery, documentation and handover session

What could go wrong

Risk	What I will do about it
The churn label is vague or inconsistent	Agree the definition in week one. If needed, rebuild the label from dates.
The data carries little signal	Say so plainly, and test a simple rule-based ranking as a fallback. A weak model does not get hidden behind a good-looking number.
A column leaks the answer	Review columns with your team and test on a later time period. A suspiciously perfect result is treated as a bug.
Offers do not change behavior	The holdout group shows it early. Stop or redesign before scaling.
Customer privacy	NDA, hashed identifiers, and deletion of the working files after handover if you ask.

13

Next steps

What happens once you say yes.

What I need from you

- The two files with the churn column filled in, or a sample of a few thousand rows to begin.
- Later, the same two files without churn, for the customers you want scored.
- A named contact who can answer questions about the data within a couple of days.
- A two-hour session in week 6 with the people who design your offers.

How we start

1	Reply to confirm the scope, or tell me what you would change.
2	We sign the NDA and you share the training files.
3	Kickoff call within a week, and the audit starts the day after.

Thank you again for your time.

Karen Maher
Independent Data Science Consultant

> *Keeping a customer costs far less than winning one back.*
> *The first step is small: share the files and agree on what counts as churn.*